

Lab-1: State Space and combinatorial explosion:

Jug X capacity = 4 litres

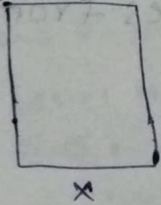
Jug Y capacity = 3 litres

1. No markings on Jug
2. Both start empty
3. And the water is unlimited.

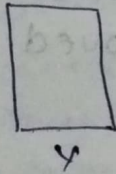
Six operations are allowed:

1. Fill X $\rightarrow$ X becomes 4
2. Fill Y $\rightarrow$ Y becomes 3
3. Empty X $\rightarrow$ X becomes 0
4. Empty Y $\rightarrow$ Y becomes 0
5. Pour X $\rightarrow$ Y $\rightarrow$ until X Empty or Y Full
6. Pour Y $\rightarrow$ X $\rightarrow$ until Y Empty or X Full

capacity = 4



capacity = 3



state (0,0)

X holds 4 litres initially,
Y holds 3 litres $\rightarrow$ X = 0 litres
Y = 0 litres

state = (0,0).

if X = 4 litres

X = 3 litres

Y = 3 litres

Y = 2 litres

state = (4,3)

state = (3,2)

$\therefore$ state is simply the water present in both jugs at that moment.

State: A state should contain only the information needed to continue solving the problem.

- def successors(state):

x, y = state

Successors:- The next state that can be reached from current state by applying one valid opn.

1. (0,0) $\rightarrow$ (4,0) $\rightarrow$ New state.

2. $(0,0) \rightarrow (0,3) \rightarrow$ New state

3. $(0,0) \rightarrow (0,0) \rightarrow$ No change

4. $(0,0) \rightarrow (0,0) \rightarrow$ No change

5. $(0,0) \rightarrow$ If X empty $\rightarrow (0,0) \rightarrow$ No change

6. $(0,0) \rightarrow$ If Y empty $\rightarrow (0,0) \rightarrow$ No change

So, the states are, $(4,0)$ & $(0,3)$

Successors of $(0,0)$

- Pour-X-to-Y = $\min(x, 3-y)$

Pour-Y-to-X = $\min(y, 4-x)$

Suppose, State = $(4,1)$

$x = 4$ litres } in the jugs.
 $y = 1$ litre

if we want to pour $x \rightarrow y$, then already y has 1 litre and its capacity is only 3 litres.

so, we can pour only $3-1=2$ litres from $x \rightarrow y$.

and only 2 litres can be moved.

if state = $(2,0)$

$x = 2$ litres } in the jugs.
 $y = 0$ litres

we can pour, $3-0=3$ litres space is there and x has only 2 litres and $\min(2,3)$ litres can be moved.

- return $[(4,y),$

$(x,3),$

$(0,y),$

$(x,0),$

$(x - \text{pour-x-to-y}, y + \text{pour-x-to-y}),$

$(x + \text{pour-y-to-x}, y - \text{pour-y-to-x})]$

From one state we can perform 6 operations.

So, we return all the possibilities from that given state.

- $start = (0,0)$
 $seen = \{start\}$
 $frontier = [start]$
 $level = 0$

$start \rightarrow$ stores the initial state of the problem
since both jugs are empty at starting,
 $start\ state = (0,0)$.

$seen \rightarrow$ a set that stores every state discovered during the search.

Initially, only one state, $(0,0)$

$seen = \{(0,0)\}$

$frontier \rightarrow$ stores the states that will be expanded at the current level of the search.

Initially, $frontier = [(0,0)]$, because the search starts from the initial state.

$level \rightarrow$ represents the depth of the search

level 0 contains the initial state

Each new level contains states reachable by one more move.

- while $frontier$ and $level < 8$:

$new = []$

first ~~two~~ conditions must be true for this:

1. $frontier \rightarrow$ are there any states left to explore?

2. $level < 8 \rightarrow$ should not go beyond level 7

because, in 2^n itself they mentioned we have to expand upto 8 levels $(0 \rightarrow 7)$

$new = [] \rightarrow$ stores only the states discovered during the current level.

Everytime the loop starts again, it is emptied.

- for s in $frontier$: $\rightarrow$ one state at a time in $frontier$
for ns in $successors(s)$: $\downarrow$,

For this state, what are all the possible states that it can reach in one move? $\downarrow$

returns 6 possible states.

- if ns not in seen:

seen.add(ns)

new.append(ns)

* basically if that new state is not there in seen then that means it is new then add that to seen and mark it as visited.

- if ns[0] == 2:

print("*** GOAL: 2L in x at level", level+1, "***")

level += 1

Print("level", level, "->", len(new), "new states:", sorted(new))

frontier = new

Print("Total states found:", len(seen))

* Two things, we have to find here:

1. run the prgrm & observe the o/p

2. A tree would predict 6, 36, 216, 1296 nodes. How many does the program actually find and what single line of code causes the difference?

- So, the main problem is to represent the water Jug problem as a state space, generate all the possible states using six operation they gave, and explore them level by level using BFS.